

RESEARCH ARTICLE

Efficient Detection of Microplastics on Edge Devices With Tailored Compiler for TinyML Applications

ALESSANDRO CERIOLI^{1,2}, LORENZO PETROSINO³, DANIELE SASSO^{3,4},
CLÉMENT LAROCHE², TOBIAS PIECHOWIAK², (Member, IEEE),
LUCA PEZZAROSSA¹, (Member, IEEE), MARIO MERONE^{5,6}, (Member, IEEE),
LUCA VOLLERO³, (Member, IEEE), AND ANNA SABATINI³, (Member, IEEE)

¹DTU Compute, Technical University of Denmark, 2800 Kongens Lyngby, Denmark

²GN Audio, 2750 Ballerup, Denmark

³Research Unit of Computer Systems and Bioinformatics, Department of Engineering, Università Campus Bio-Medico di Roma, 00128 Rome, Italy

⁴Department of Biology, University of Naples Federico II, 80138 Naples, Italy

⁵Research Unit of Intelligent Health Technologies, Department of Engineering, Università Campus Bio-Medico di Roma, 00128 Rome, Italy

⁶Fondazione Policlinico Universitario Campus Bio-Medico di Roma, 00128 Rome, Italy

Corresponding author: Mario Merone (m.merone@unicampus.it)

The work of Alessandro Cerioli was supported by the European Union's Horizon Research and Innovation Program under Grant 101070374. The work of Anna Sabatini was supported by Programma Operativo Nazionale (PON) "Ricerca e Innovazione" 2014–2020.

ABSTRACT The current study aims to train and benchmark AI models tailored for the detection of microplastic in water from scattered signals. We trained two different models, the first based on a Multi-Layer Perceptron (MLP) and the second on a Gated Recurrent Unit (GRU). A Neural Architecture Search algorithm was used to determine the optimal configuration for each of the two models. Moreover, for deployment on edge devices, a specific custom-made compiler was designed and used. The compiler is specifically designed for TinyML applications and, therefore, for resource-constrained devices. It bypasses traditional inference engines, compiling the NNs to native C code using only standard C libraries. Our approach demonstrated better performance compared to state-of-the-art frameworks such as ONNX Runtime, achieving better latency, memory usage, energy consumption, and a higher portability. This highlights the potential of our method for efficient and effective microplastic detection in environmental monitoring.

INDEX TERMS Microplastic detection, compiler, TinyML, quantization.

I. INTRODUCTION

Microplastics have grown exponentially in recent years, becoming an increasingly worrying topic for humanity and the environment [1]. Microplastics are synthetic solid particles or polymeric matrices with regular or irregular shapes and dimensions between 1 μm and 5 mm insoluble in water [2].

Due to their unique characteristics, such as being thermoplastic and thermosetting, microplastics are resistant to biodegradation and exhibit very slow degradation rates, enabling them to have a dramatic impact for many years.

The associate editor coordinating the review of this manuscript and approving it for publication was Muammar Muhammad Kabir¹.

Today, microplastics are ubiquitous. They can be found in marine, freshwater, and terrestrial ecosystems [3], [4], [5].

Microplastics' detection in water is critically important to address one of the 17 Sustainable Developing Goals adopted by all the United Nations Member States in 2025 [6]: conserve and sustainably use the oceans, seas, and marine resources for sustainable development. Microplastics represent a threat to aquatic ecosystems and human health, which requires an advanced technological solution for their detection and removal. The development of efficient sensing techniques, such as advanced spectroscopy techniques that include Fourier transform infrared spectroscopy (FTIR) and Raman spectroscopy [7], in conjunction with the improvement of artificial intelligence models, leads to providing a robust framework for monitoring water quality [8], [9], [10].

In this way, it is possible to integrate Artificial Intelligence of Things (AIoT) that ensures real-time data acquisition and analysis through massive IoT sensors and communication networks [11]. The ability to perform high-level tasks such as pattern recognition and decision-making on edge devices further enhances the feasibility of deploying these systems in remote and resource-constrained environments, and by using energy-efficient solutions, the carbon footprint associated with such monitoring activities can be significantly reduced.

This is where TinyML comes into play, a growing field that deals with bringing machine learning models to very small devices with limited resources, such as microcontrollers and sensors [12]. This is especially important for the IoT, where several connected devices have to operate with few resources in terms of computational power and memory. Therefore, TinyML is the research field related to the development of techniques to compress neural networks (NNs) and deploy them on resource-constrained devices [13], [14]. Examples of TinyML applications include smart home devices [15], wearable health monitors [16], and anomaly detection in industrial automation systems [17], where processing data locally on small devices reduces latency and increases privacy [18], [19].

To achieve this, *compilers* are fundamental software tools that enable the compilation and deployment of NNs on different hardware architectures. They transform high-level models into executable code optimized for specific hardware platforms, ensuring efficient use of available resources, such as optimized memory management. Examples of optimizations include removing redundant connections (**pruning**) [20], reducing the precision of models' weights (**quantization**) [21], and combining several operation nodes into a single one (**node fusion**) [22]. Frameworks designed for embedded systems include TensorFlow Lite (TFLite [23]), Apache TVM [24], [25], and ONNX Runtime [26]. These tools are crucial for converting complex NNs into lightweight versions suitable for low-power devices, enabling the practical use of AI in everyday objects.

Quantization, in particular, is a technique that reduces the precision of the data types used to represent the parameters of a model, typically from 32-bit floating-point numbers to 8-bit integers. This not only reduces the size of the model but also speeds up inference and decreases power consumption, making it ideal for IoT applications. Although it entails a small loss of accuracy, quantization allows NNs to run efficiently on resource-constrained devices, extending the battery life of portable devices and improving response times.

Despite its advantages, quantization is still not integrated into many frameworks designed to compile NNs. This lack poses a problem for developers and ML engineers and represents an important limitation for the integration of NNs on embedded systems. To solve this problem, we propose a new framework based on the ONNX API, to compile high-level quantized models into native C code. This approach

aims to simplify the deployment process, ensuring a high portability of the models across a wide range of devices with limited resources.

Therefore, with the current article, we introduce a different approach for the deployment of NNs for microplastic detection tasks using a straightforward compilation of the models to C code, rather than relying on inference engines as in the case of ONNX Runtime or TFLite. This leads to notable improvements in several aspects:

- Greater network responsiveness, with significantly lower latency times;
- Less memory space required by removing the inference engine;
- Greater portability, as not all devices are compatible with inference engines;
- Lower energy consumption.

These are essential performance improvements for applications such as microplastic detection, where continuous device use with low latency times is required. Therefore, the main contributions of this article are:

- Introduction to a new framework to convert ONNX format to native C code, supporting quantization;
- Framework application on a specific use case: microplastic detection;
- Benchmark of the algorithms on several MCU.

The organization of the articles follows a top-down structure starting with Section II in which the *Background and Related Works* are reported, emphasizing the deployment of NNs on hardware architectures, the linear quantization and grid search. Successively, Section III describes the *Materials and Methods*, where we analyze the pipeline and features of NeuralCasting compiler, the integration of quantization, and provide details on the dataset, models, devices used, and the experimental setup for the microplastic detection task. Section IV presents our *Results and Discussion*, reporting an analysis of our findings. Finally, Section V concludes the paper.

II. BACKGROUND AND RELATED WORKS

A. DETECTION OF MICROPLASTICS WITH NN

Recent advancements in the detection of microplastics include the use of neural networks to enhance accuracy and efficiency. These algorithms are applied starting from several data types. For example, one approach combines UV light imaging with deep learning, utilizing a Faster-RCNN model with a ResNet-50-FPN backbone. This method achieved a precision rate of 85.5–87.8% in classifying various polymer types from microplastic images, significantly reducing the time and cost associated with traditional laboratory methods [27]. Another innovative technique employs a neural network algorithm called SpecATNet, which utilizes surface-enhanced Raman spectroscopy (SERS) for simultaneous separation and detection of multiple microplastic types. This method allows for rapid analysis directly from environmental samples without extensive pretreatment, showcasing

the potential of machine learning in environmental monitoring [28]. Additionally, research has demonstrated the effectiveness of neural networks for microplastic segmentation, optimizing their structure to improve performance while minimizing computational demands [29].

These results demonstrate the increasing of the application NN in this specific task regardless of the type of sensor used for detection.

B. DEPLOYMENT OF NNS ON HW ARCHITECTURES

Deploying NN models on hardware architectures is a fundamental step to making these technologies available to a wide range of customers in everyday life. Frameworks designed for this purpose play a relevant role by optimizing models to leverage the specific capabilities of different hardware, ensuring high efficiency and performance. Moreover, these frameworks are used to improve the portability of the NNs, allowing them to be deployed across a wide range of devices and hardware platforms with few high-level modifications. The optimizations introduced by the compilers are fundamental in reducing essential metrics for deployment in embedded systems, such as reduced inference time, low energy consumption, and less memory usage for weight storage. These are all fundamental features of TinyML applications.

ONNX (Open Neural Network Exchange) [30] and ONNX Runtime are powerful tools to store data related to NNs and facilitate their deployment across diverse hardware environments, respectively. ONNX is an open-source format for representing deep learning models designed to promote interoperability between different frameworks. Instead, ONNX Runtime is an inference engine that executes the models using ONNX format as input. It provides different specific optimizations for various hardware platforms. ONNX Runtime implements quantization techniques to convert floating-point models into integer models, which reduces the model size and computational requirements while maintaining a high accuracy. In particular, quantization occurs via operators in Q Format, which are interpreted by ONNX Runtime specifically to provide a quantized implementation. For example, the Gemm operator in ONNX provides a generic General Matrix Multiplication in floating point. Its quantized counterpart is the QGemm unit, which has further attributes related to quantization, such as zeros and scaling factors.

TensorFlow and TFLite are among the most popular frameworks for developing and deploying NN models on embedded systems. TensorFlow is an open-source library developed by Google and designed for deep learning applications. It provides a package for NN design, training, and deployment on various hardware platforms. TFLite uses TensorFlow models to convert them into a lighter format suitable for resource-constrained devices to improve metrics such as latency time, energy consumption, and storage. TFLite also supports model quantization via specific API functions that allow us to determine hyperparameters such

as the target data type (generally signed or unsigned 8-bit integers).

The fundamental difference between our approach compared to the other frameworks described, representing the state of the art, is the pipeline adopted to deploy the models on the devices. Indeed, we used NeuralCasting [31], [32], a compiler proposed by us described in section III-A, specifically designed to deploy small NNs on embedded systems and resource-constrained devices. NeuralCasting is not based on an inference engine as ONNX Runtime or TFLite but leverages a direct compilation of the model into native C code. It allows us to limit the abstraction introduced by the inference engine, which also showcases, in addition to a higher memory usage required, an additional latency for the ONNX format interpretation, which can be relevant in terms of performance on devices with limited resources. Furthermore, the native C code is based exclusively on the standard C libraries, extending portability and the possibility of deployment across multiple architectures.

C. LINEAR QUANTIZATION

Linear quantization is an optimization technique for neural networks that involves reducing the bit-width of the data representation, usually from full-precision floating point 32 bits to 8-bit integers. This method maps a continuous range of floating point values to a discrete set of integers. The mapping occurs through a linear transformation, i.e., within the scaling and shifting of the original range and approximating on discrete levels. This process reduces the memory space required for NN inference, improving memory management and optimizing the cache efficiency, leading to an improvement in inference time as well.

The main advantage of using linear quantization is the ability to employ 8-bit precision (or even 4-bit) integer values for both inference and storage. Consequently, this approach exploits the use of integer types both to reduce the memory storage required and to exploit integer operations, computationally less expensive than floating-point operations. On the other hand, linear quantization allows a linear distribution of discrete levels, which is sometimes not optimal for representing the distribution of weights, e.g., after a fine-grained pruning process, where low-magnitude weights are less frequent. Moreover, as with most of the compression techniques, quantization introduces an approximation that leads to an accuracy drop, compared to the original 32-bits floating point model. The performance benefits of linear quantization can vary depending on the hardware and the specific NN architecture. Some models may experience a significant drop in accuracy, requiring a careful evaluation of the trade-offs for each specific application.

Linear quantization can be formalized as follows:

$$\mathbf{W} = (\mathbf{Q} - z) \cdot s \quad (1)$$

where $\mathbf{W}$ is the weights matrix of the 32-bit floating point model, $\mathbf{Q}$ is the quantized weights matrix, for example 8-bit integers. Finally, z and s are the *zero point* and the *scaling*

factor of the linear transformation, respectively. Therefore, z represents the shifting, and s represents the scaling.

The scaling factor s can be calculated starting from a representative dataset for the activations and from the model matrices for the weights, following the equation:

$$s = \frac{r_{max} - r_{min}}{q_{max} - q_{min}} \quad (2)$$

where r_{max} is the maximum floating point value in the weights matrix and r_{min} is the minimum value. Instead, q_{max} and q_{min} are the extremes of the representable range using N -bits integers, i.e., $[-2^{N-1}, 2^{N-1} - 1]$ for signed integers or $[0, 2^N - 1]$ for unsigned integers.

The zero point can be calculated accordingly from the Equations 1 and 2:

$$z = \text{round} \left(q_{min} - \frac{r_{min}}{s} \right) \quad (3)$$

D. NEURAL ARCHITECTURE SEARCH

Neural Architecture Search (NAS) is a process of automating the design of NNs, privileging the most suitable architectures for specific tasks. It is a method widely used in TinyML, where metrics such as latency, memory footprint, and energy consumption are considered crucial for efficient deployment on resource-constrained devices. For example, it is possible to determine the trade-off between the optimal bit-width of the quantization and the model's accuracy through NSGA-II evolutionary algorithms [33]. Moreover, the automatic network design was demonstrated to be faster and more accurate than manual design by experts [34]. NAS algorithms consist of three primary components: the search space, the search strategy, and the performance evaluation. The search space is the set of candidate architectures, the search strategy is the algorithm used to navigate through the search space (e.g., evolutionary algorithms or reinforcement learning), and the performance evaluation is the objective function to evaluate the fitness of a candidate architecture.

One of the most adopted NAS algorithms is the Grid Search, where the search space is a grid of N dimensions (considering N as the number of hyper-parameters to estimate), and each architecture of the search space is evaluated. This simple approach guarantees finding the best solution for the search space; nonetheless, it is computationally expensive and suitable for small-sized NNs, such as architectures proposed for TinyML applications.

III. MATERIALS AND METHODS

A. NEURALCASTING COMPILER

In this section, we introduce NeuralCasting, a Python-based compiler designed to convert ONNX format models into native C code. This approach optimizes the deployment of NNs on edge devices, particularly bare metal boards with limited memory and computational power, where the lack of an operating system complicates the deployment. Frameworks like Google's TFLite and Microsoft's ONNX Runtime allow the deployment of NNs on embedded

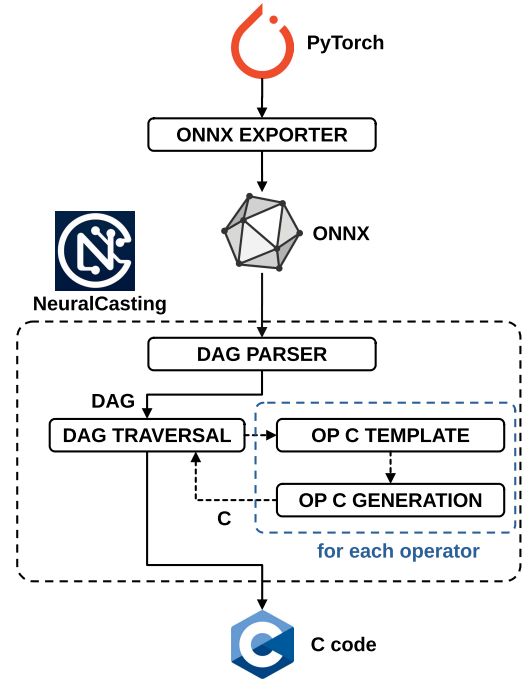


FIGURE 1. NeuralCasting parses ONNX into the resulting DAG, and traverses it to generate C code from templates for each operator. This process emits optimized C implementations from the neural network model.

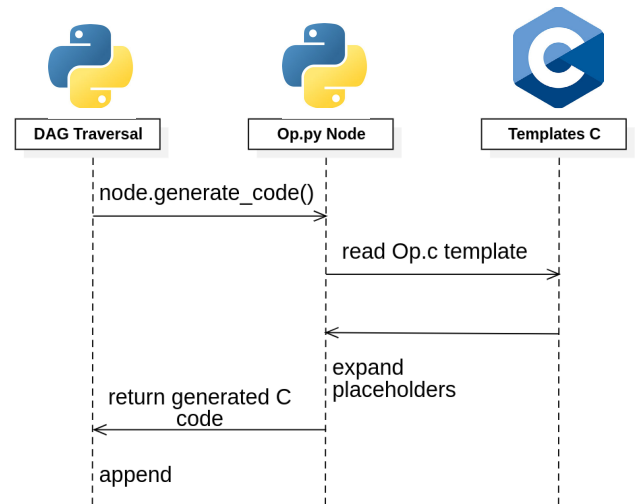


FIGURE 2. The workflow of NeuralCasting for the C code generation. The kernel is implemented in Python. During the DAG traversal, native C code is generated from templates. Indeed, the templates are expanded using the information stored in Op.py (acquired from the original ONNX model). Finally, the C code generated for the operator is appended to the current generated code of the entire network.

systems; nevertheless, they require an inference engine, introducing an overhead in memory usage and inference time significant on edge devices. Moreover, compatibility issues often arise on bare metal microcontrollers, where an additional abstraction layer is undesirable and challenging to deploy.

The overall architecture of NeuralCasting is illustrated in Figure 1. In the following, we describe its main components and underlying structure. NeuralCasting's architecture utilizes the ONNX format as its foundation, i.e., a Direct Acyclic Graph (DAG). In ONNX, nodes take the form of tuples containing the node's name, the name of the incoming edges, and the name of the outgoing edges. NeuralCasting parses these nodes, converting them into an internal data structure holding the metadata of the operators and connecting the network using memory references. This enables graph traversal and sequential C code generation based on the operator type (e.g., Gemm, ReLU, Convolution) and metadata (e.g., the kernel size of the Convolution).

The C code generation occurs through the information stored inside the internal DAG, used as an intermediate representation by the compiler. Indeed, the graph traversal occurs, with each node containing information such as the operator type, the types of data used as input and output, and other attributes specific to the operator (e.g., kernel size for a convolutional layer). Successively, the kernel accesses a hand-crafted C template, univocally related to the operator. The templates contain *placeholders* to expand into generated C code, ranging from simple macros to more complex sections of code, such as indexing to ensure broadcasting between matrices and tensors of different sizes. The information to expand the placeholders is inside the node. Finally, the kernel appends the generated C to the main generated code, created from the previous operators analyzed during graph traversal. Figure 2 shows the pipeline used by the compiler for generating C code.

NeuralCasting provides flexibility in memory allocation regarding the weights and biases of the models, allowing allocation either in the process's data segment or the heap, depending on requirements. It relies on a compiler flag `-alloc`, which can be set to `heap` for heap allocation or `data` for data segment allocation as global variables. Concerning the allocation in the heap, C code and binary files containing weights and biases are generated and read within the program through a specific `nm_alloc` function that allocates memory in the heap. Similarly, an `nm_free` function deallocates memory. To prevent memory fragmentation, allocate weights once at the program's start and deallocate them at the end. For data segment allocation, weights are compiled internally in the data segment, eliminating the need for external sources. Considering our microplastic detection task, which demands high performance with relatively small matrices, we chose to allocate in the data segment to maximize performance. Moreover, deploying on bare metal chips without an operating system introduces additional difficulties and risks related to manual dynamic memory allocation.

Also, NeuralCasting provides the generation of parallelized C code using *OpenMP*, through a `-parallel` flag settable to `omp`, for the automatic generation of OpenMP clauses. Furthermore, a `-n_threads` flag allows to define the number of threads used. Considering our small-sized models, using a single thread was optimal, as parallelization

did not enhance performance due to the overhead of thread management.

Compared to other state-of-the-art frameworks like *TFLite* and *ONNX Runtime*, NeuralCasting currently offers fewer optimizations. It lacks inter-layer optimizations, node fusion, and architecture-specific optimizations. However, NeuralCasting generates code for bare metal edge devices, making it optimal for small models where advanced parallelization and vectorization are unnecessary. Generating native C code enhances portability across architectures, and the absence of an inference engine eliminates overhead, improving performance in terms of both memory usage and inference time. Nevertheless, tiling or vectorization tailored for diverse architectures is a valuable improvement in future work.

B. INTEGRATION OF QUANTIZATION IN NEURALCASTING

Due to the growing need to compress models and optimize performance on edge devices, NeuralCasting presents an extension to support ONNX quantization units, such as *QGemm*, *QAdd*, *QSigmoid*, and other fundamental components of quantized NNs.

A quantized ONNX model can be exported via the ONNX API, using a representative dataset and performing post-training quantization. The ONNX Q units provide necessary quantization data, such as scaling factors and zero points.

Supporting quantization in a compiler significantly enhances the deployment of machine learning models on resource-constrained devices. It reduces model size and computational requirements, resulting in faster inference, lower latency, and improved performance, essential for real-time applications. Additionally, quantized models consume less energy, extending battery life in portable devices. Integrating quantization into a compiler allows developers to focus on model development while ensuring efficient, high-performance deployment across various hardware platforms.

Quantization was integrated into NeuralCasting similarly to the integration of traditional float operators. Indeed, NeuralCasting is based on the ONNX format, generating native C code using the information inside each unit. ONNX provides units specifically designed for quantized operators, named Q-Units. For example, *QLinearConv*, *QLinearMatMul* and *QGemm*. Moreover, it provides a specific *QuantizeLinear* Q-Unit for the quantization of weights and activations, and a Q-Unit for de-quantization (i.e., to re-convert the integers to floating point), called *DequantizeLinear*. These units present specific attributes for quantization, such as zero points and scaling factors. The quantization hyperparameters are used by NeuralCasting to generate C code for the quantized operators. Figure 4 reports a Quantized MLP model, where the quantized input feeds a series of Quantized Generalized Matrix Multiplications (*QGemm*). Finally, the resulting quantized tensor is de-quantized to provide the float32 output tensor. As presented in Figure 4, each quantized operator provides the necessary scaling factors and zero points for the inputs and outputs. For example, considering a *QGemm*

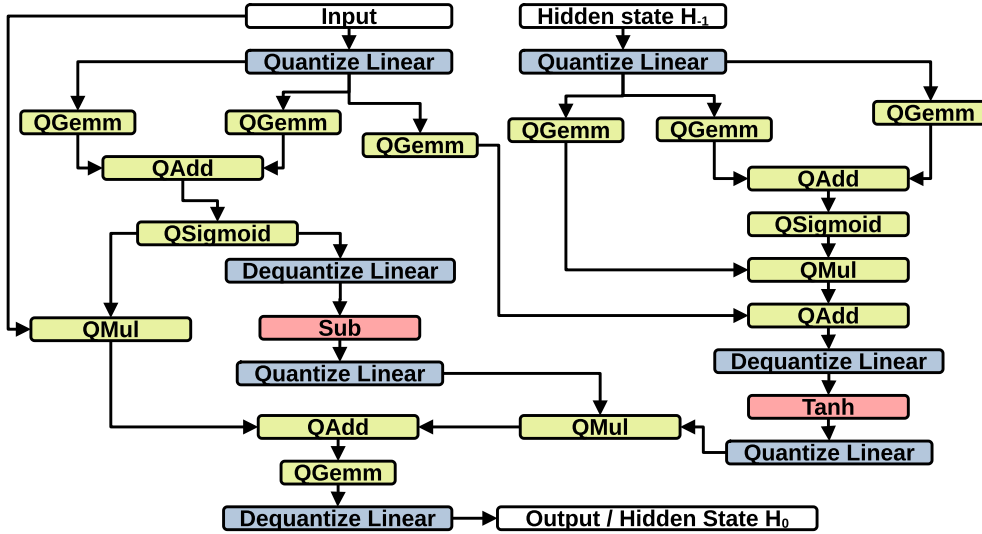


FIGURE 3. The model based on quantized GRU, re-implemented on its single elements. Green nodes represent quantized operators, red nodes indicate floating-point operators, blue nodes correspond to quantization and dequantization operations, and white nodes represent the model's inputs and outputs.

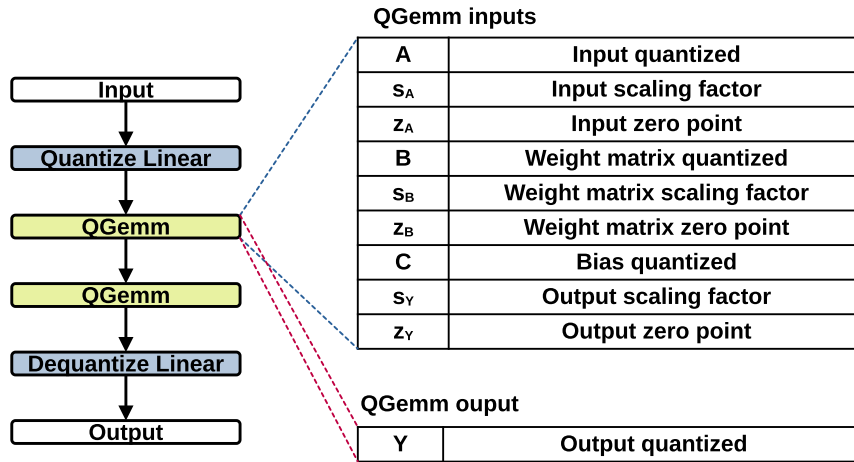


FIGURE 4. The ONNX graph of the Quantized MLP model. The QGemm, as a Q-Unit, presents attributes specific to linear quantization, consequently used by the compiler for generating C code.

operator, the linear quantization used applies the linear quantization definition 1 to a floating point Gemm defined in 6, resulting in the equation:

$$\mathbf{q}_y = \frac{s_W \cdot s_x}{s_y} \cdot (\mathbf{q}_W \cdot \mathbf{q}_x + \mathbf{q}_{\text{bias}}) + z_y \quad (4)$$

with the pre-computed factor $\mathbf{q}_{\text{bias}}$:

$$\mathbf{q}_{\text{bias}} = \mathbf{q}_b - z_x \cdot \mathbf{q}_W \quad (5)$$

where s_W is the scaling factor of the weights matrix, s_x is the scaling factor of the input tensor, s_y is the scaling factor of the output, $\mathbf{q}_W$ is the quantized weights matrix, q_x is the quantized input tensor, z_y is the zero point of the output tensor, $\mathbf{q}_b$ is the quantized bias vector and z_x is the zero-point of the input tensor.

ONNX does not provide a quantized unit for the GRU. As a result, it was implemented in NeuralCasting by combining

elementary operators based on the equation 7 (see Figure 3). Finally, lookup tables based on 256 points have been implemented for the activation functions to improve inference time.

C. DATASET

A synthetic dataset has been generated according to [35]. The data simulate the signals generated by the particle traversal through the measurement system over time, approximating them as Gaussian functions. Each Gaussian function within the dataset is characterized by distinct parameters such as mean and standard deviation mapped to capture the variability in the sizes of microplastics that may be present in the flow. We added noise to the signal to simulate potential measurement interference, mimicking real-world conditions in which extraneous factors can affect the accuracy of the

readings. The signal depends on several parameters: the signal amplitude variation ($\mathcal{A}$), the signal-noise ratio (SNR), and the a priori probability (P_1) of finding the particle in the signal. Each signal has a length of 41 samples and is labeled with 0 or 1, indicating the presence or absence of the particle. Each realization of the dataset is composed of 1000 signals resulting in a matrix of shape 1000×41 and a ground-truth vector with a dimension 1000×1 . In this paper, all the generated signals have SNR equal to 0.05, $P_1 = 0.5$ and $\mathcal{A} \in [0.6, 1.4]$, by varying these parameters, it is possible to reconstruct a realistic range of particle sizes that reflects the natural heterogeneity of microplastics.

D. MODELS DESCRIPTION

We employed two distinct NN architectures to analyze and process our data: Multilayer Perceptrons (MLPs) and Gated Recurrent Units (GRUs). These models were selected due to their proven effectiveness in handling different data types. Below, we provide a detailed description of each model, including its structure, functioning, and the specific methods used for training.

- **Multilayer Perceptrons (MLPs):** MLPs represent a foundational class of artificial NNs extensively utilized for a wide range of pattern recognition and classification tasks. An MLP is structured with an input layer, one or more hidden layers, and an output layer, where each node, apart from the input nodes, functions as a neuron equipped with a non-linear activation function. This layered architecture enables MLPs to approximate complex, non-linear functions, making them particularly effective in scenarios that require the learning of complex patterns and relationships within data. The training of MLPs is typically conducted using back-propagation, a gradient-based optimization technique that minimizes the error between the predicted and actual outcomes. The output of a neuron in an MLP can be represented as:

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right) \quad (6)$$

where f is the activation function, w_i are the weights, x_i are the inputs, and b is the bias. Despite their relatively simple architecture compared to more advanced models, MLPs remain a cornerstone in the field of machine learning due to their robustness and versatility in handling diverse datasets.

- **Gated Recurrent Units (GRUs):** GRUs are a specialized form of recurrent neural networks (RNNs) designed to address some of the limitations inherent in traditional RNNs, such as the vanishing gradient problem [36]. GRUs incorporate gating mechanisms. Specifically, *update* and *reset* gates, which modulate the flow of information across the network. This allows GRUs to maintain and update information over long sequences more effectively, making them highly

suitable for tasks involving sequential data, such as time-series forecasting and natural language processing. The equations describing a GRU are:

$$\begin{aligned} \mathbf{z}_t &= \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z) \\ \mathbf{r}_t &= \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r) \\ \tilde{\mathbf{h}}_t &= \tanh(\mathbf{W}_h \mathbf{x}_t + \mathbf{r}_t \odot \mathbf{U}_h \mathbf{h}_{t-1} + \mathbf{b}_h) \\ \mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \end{aligned} \quad (7)$$

where $\mathbf{z}_t \mathbf{z}_t$ is the update gate, $\mathbf{r}_t$ is the reset gate, $\tilde{\mathbf{h}}_t$ is the new hidden state, $\mathbf{h}_t$ is the final hidden state, $\mathbf{W}$ and $\mathbf{U}$ are weight matrices, $\mathbf{b}$ is the bias vector, concatenation of $(\mathbf{b}_z; \mathbf{b}_r; \mathbf{b}_h)$, and $\odot$ denotes element-wise multiplication. The streamlined architecture of GRUs facilitates faster training times and reduces computational demands while still providing competitive performance. The ability of GRUs to capture temporal dependencies with greater efficiency underlines their importance in modern NN applications, particularly in contexts where data exhibits sequential or temporal characteristics.

E. EXPERIMENTAL SETUP

A NAS process was executed to select the network designated as the final model to be trained and deployed on devices, as described in II-D.

We used the Adam optimizer with a learning rate of 0.001 to train the models. The training was performed using a batch size of 64 for 10 epochs, employing 5-fold stratified cross-validation to ensure robust performance evaluation. Given the resource-constrained nature of TinyML applications, we applied practical search space constraints. Memory constraints were considered to ensure that only small architectures were selected, making them deployable on MCUs. Computational efficiency was also a key factor, as the chosen models needed to maintain low latency and low energy consumption on edge devices.

For the MLP, a greedy grid-search approach was applied, considering the number of layers (1, 2, 3) and the number of neurons per layer (8, 16, 32, 64) as parameters. Similarly, for the GRU, the same approach was applied to determine the most effective hidden size (8, 16, 32, 64).

The final models were chosen based on their classification accuracy on the microplastic detection task while ensuring real-time inference on low-power devices. In cases where multiple architectures achieved similar accuracy, the model with the least complex architecture was preferred to optimize the trade-off between accuracy and computational cost.

The steps involved included:

- **Parameter Initialization:** Setting the initial values for the number of layers and neurons per layer as per the grid search ranges;
- **Model Generation:** Constructing various network architectures by systematically varying the number of layers and neurons;
- **Training:** Each generated model was trained using the designated dataset;

- **Evaluation:** The performance of each model was assessed based on specific metrics, i.e., accuracy and computational efficiency. In cases where multiple models achieved similar accuracy, the architecture with the lower computational cost was chosen;
- **Selection:** The model with the optimal performance metrics was selected as the final architecture for deployment.

This approach ensured a comprehensive exploration of potential network configurations, allowing the identification of the most effective model for the given application.

Once we identified the architectures designated for porting to edge systems, we proceeded to quantize them. We chose to perform float32 to int8 quantization using either the native functions provided by ONNX Runtime and via NeuralCasting.

The models were quantized via Post-Training Quantization (PTQ), i.e. the quantization was carried out at the end of the model training phase. Usually, an approach via Quantization Aware Training (QAT) is preferable, as including the quantization process within the training allows a higher accuracy recovery. Nonetheless, in this case, the PTQ was preferred for several reasons. First, the microplastic detection task relies on a binary classification, which is little affected by small approximations of the output tensor. Consequently, the difference between the accuracy via PTQ and QAT is negligible. Furthermore, QAT is notoriously expensive from a computational point of view as it involves a simulation of the quantized model in the forward pass of training. Considering an expensive NAS algorithm such as Grid Search, this approach would have proven to be unnecessarily expensive from a computational point of view, with minimal gains in terms of accuracy.

After quantizing the models, we conducted 100 tests using 100 synthetically generated datasets (according to parameters reported in Section III-C) to compare the performance of both the original and quantized versions of the MLP and GRU models. This comparison was performed by implementing a Student's t-test on the populations of the accuracies obtained from the 100 tests between the original models and their quantized version. Moreover, the comparison in terms of accuracy between the two quantized versions of the model was not carried out since the two models perform the same mathematical operations exploiting the same quantization ranges as explained in subsection III-B, thus resulting in two perfectly equal models at the level of inference results.

F. DEPLOYABILITY ON MCU

The quantized versions of the MLP and GRU models were tested on edge devices. In particular, the C implementation of the code is flashed into a set of microcontrollers with characteristics that cover a varied performance range. The following MCUs were chosen: STM32H747XI-M7/M4 with a dual-core ARM Cortex-M/ at 480 MHz and Cortex-M4 at 240 MHz processor, nRF52840 with an ARM Cortex-M4

CPU with FPU running at 64 MHz processor, and ESP32-PICO-D4 with a dual-core Xtensa 32-bit LX6 microprocessor at 240 MHz. We tested the microcontrollers using their respective development boards: Portenta H7 for the STM32H747XI, Arduino Nano 33 BLE for the nRF52840, and M5StickC Plus for the ESP32-PICO-D4. Tests were also performed on a single-board computer, Raspberry Pi 4, which has a Broadcom BCM2837B0 processor, Cortex-A53 (ARMv8) 64-bit SoC, with a frequency of 1.4 GHz. The technical specifications of the devices are shown in Table 1. Each microcontroller and single-board computer was chosen to represent a broad spectrum of processing power and memory capabilities, providing a comprehensive assessment of the model's scalability and adaptability across different hardware platforms. For each MCU, inference was performed on both models 10,000 times, and the execution time of each inference was measured. Finally, the mean, maximum, and standard deviation of the execution times were calculated. To better compare performance on different device power consumption, memory occupation was evaluated.

IV. RESULTS AND DISCUSSION

In our case, the NAS process resulted in the best parameters identified for the MLP being one layer and eight neurons. Instead, the optimal hidden size for the GRU was determined to be eight. After quantization, 100 tests were performed to compare the original and quantized models: the original MLP model and the quantized MLP model achieved the same average accuracy of 0.991 ± 0.001 . The conducted Student's t-test indicated that the difference between the distributions was not significant (p -value = 0.733). For the GRU, the original model showed an average accuracy of 0.993 ± 0.001 , whereas the quantized model demonstrated a slight drop in accuracy (0.985 ± 0.001). This difference was statistically significant (p -value < 0.05). Quantization techniques such as quantization-aware training (QAT) use the training phase to estimate the hyperparameters scaling factors and zero points, generally obtaining superior performance to post-training quantization (PTQ), which instead estimates the hyperparameters after training via a representative dataset. However, in our specific case, the PTQ has a very high accuracy, so using the QAT would be negligible, if not counterproductive, considering the higher computational effort required compared to the PTQ.

Upon deploying the software on the different devices chosen, the results in terms of power consumption, memory occupation, and inference time were evaluated. Considering the power consumption of the algorithms, as shown in Table 1, it can be seen that there is a variation based on the characteristics of the chosen platform. For the sake of brevity, the consumption figures in a single column for both the GRU and the MLP implementation as the values were found to be identical. The lower power consumption during the task execution is around 0.02 W for the Arduino Nano 33 BLE Sense platform, while the higher consumption is reached by the M5StickC Plus, which is 0.44 W. It was seen, as shown

TABLE 1. Technical Specification of the devices tested powered at 5V.

Board	MCU	Memory	Frequency [MHz]	Current [A]	Power [W]	Voltage [V]
Portenta H7	STM32H747XI-M7	2MB Flash, 1MB RAM	480	0.12	0.6	5
	STM32H747XI-M4		240	0.12	0.6	5
Arduino Nano 33 BLE	nRF52840	1MB Flash, 256KB RAM	64	0.02	0.1	5
M5StickC Plus	ESP32-PICO-D4	4MB Flash, 520KB SRAM	240	0.5	2.5	5
Raspeberry Pi 4	SoC BCM2711	8GB LPDDR2 SDRAM	1800	0.48	2.4	5

TABLE 2. Performance in terms of latency time relating to the MLP model, using `ONNX Runtime` and `NeuralCasting` for deployment, on different HW architectures. The acronym N.A. means *Not Available*, i.e. the framework is not supported by the platform.

Hardware Platform	Inference Time [μs]		Maximum Time [μs]	
	NeuralCasting	ONNX Runtime	NeuralCasting	ONNX Runtime
x86_64 Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz	0.63 ± 0.59	32.20 ± 4.97	400.00	624.00
Portenta H7 - STM32H747XI -M7	46.28 ± 3.01	N.A.	56.00	N.A.
Portenta H7 - STM32H747XI -M4	46.16 ± 2.91	N.A.	54.00	N.A.
Arduino Nano 33 BLE	407.62 ± 19.51	N.A.	474.00	N.A.
M5StickC Plus	63.55 ± 1.86	N.A.	71.00	N.A.
Raspeberry Pi 4	6.72 ± 2.29	45.85 ± 1.81	33.00	87.97

TABLE 3. Performance in terms of latency time relating to the GRU model, using `ONNX Runtime` and `NeuralCasting` for deployment, on different HW architectures. The acronym N.A. means *Not Available*, i.e. the framework is not supported by the platform.

Hardware Platform	Inference Time [μs]		Maximum Time [μs]	
	NeuralCasting	ONNX Runtime	NeuralCasting	ONNX Runtime
x86_64 Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz	1.10 ± 0.90	13.20 ± 1.76	133.00	63.20
Portenta H7 - STM32H747XI -M7	211.60 ± 3.86	N.A.	247.00	N.A.
Portenta H7 - STM32H747XI -M4	211.89 ± 3.95	N.A.	252.00	N.A.
Arduino Nano 33 BLE	1964.81 ± 11.16	N.A.	2001.00	N.A.
M5StickC Plus	255.59 ± 3.35	N.A.	262.00	N.A.
Raspeberry Pi 4	9.74 ± 3.71	109.42 ± 3.53	55.00	194.78

TABLE 4. Performance in terms of latency time relating to the MLP model and GRU model, using `Tensorflow Lite` for deployment, on different HW architectures.

Hardware Platform	MLP - Tensorflow Lite		GRU - Tensorflow Lite	
	Inference Time [μs]	Maximum Time [μs]	Inference Time [μs]	Maximum Time [μs]
Portenta H7 - STM32H747XI -M7	78.52 ± 2.19	90	34 ± 182	1000
Portenta H7 - STM32H747XI -M4	78.32 ± 1.98	88	34 ± 184	1000
Arduino Nano 33 BLE	740.16 ± 12.64	764	308 ± 462	1000
M5StickC Plus	40.01 ± 1.57	48	60 ± 237	1000

in Table 5, that the RAM and Flash used are a bit higher for the GRU model compared with the percentage required for the MLP for all the platforms. Overall, the STM32H747XI-M7 had the lowest resource usage, whereas the nRF52840 exhibited the highest RAM consumption, indicating that both models show consistent resource usage patterns across different microcontrollers, which is crucial for optimizing model deployment in resource-constrained environments.

In addition, it is worth mentioning that directly compiling the network in C is advantageous compared to using an inference engine on embedded systems. Besides the additional abstraction layer, the inference engine requires no negligible memory space on small edge devices. Instead,

directly compiling the network into C requires only the storage relating to the weights and biases of the model and the algorithms of the operators, respectively, in the data segment and the text segment of the program.

A more in-depth analysis is required for the evaluation of inference.

Table 2 shows the results for the quantized MLP model in terms of mean inference time, standard deviation, and maximum time. Where possible, results for the Neural Casting and `ONNX Runtime` implementation were compared. Using the native C code generated by `NeuralCasting`, there is an evident increase in inference time with the degradation of the hardware characteristics. The result achieved with

TABLE 5. Resource Utilization of GRU and MLP Models on various Microcontrollers, for each cell, is reported the ratio between the used MB and the total available MB for both the Neural Casting and Tensorflow Lite implementation.

MCU	MPL				GRU			
	Neural Casting		tensorflow lite		Neural Casting		tensorflow lite	
	RAM [MB/MB]	Flash [MB/MB]	RAM [MB/MB]	Flash [MB/MB]	RAM [MB/MB]	Flash [MB/MB]	RAM [MB/MB]	Flash [MB/MB]
STM32H747XI -M7	0.14/0.52	0.14/0.79	0.15/0.52	0.41/0.79	0.15/0.52	0.15/0.79	0.16/0.52	0.41/0.79
STM32H747XI -M4	0.12/0.29	0.07/1.04	0.13/0.29	0.35/1.04	0.12/0.29	0.08/1.05	0.14/0.29	0.35/1.05
nRF52840	0.12/0.26	0.08/0.98	0.13/0.26	0.35/0.98	0.12/0.26	0.08/0.98	0.14/0.26	0.35/0.98
ESP32-PICO-D4	0.10/0.33	0.27/1.31	0.11/0.33	0.58/1.31	0.10/0.33	0.27/1.31	0.12/0.33	0.58/1.31

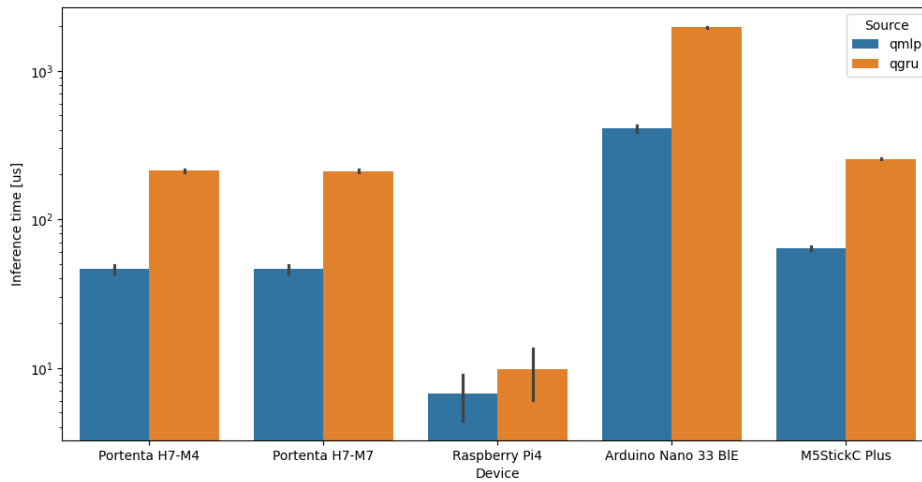


FIGURE 5. The latency of the MLP and GRU models on different hardware platforms are reported in the current bar chart, where blue bars showcase the average and standard deviation related to the quantized MLP, while orange bars refer to the quantized GRU model. For each microcontroller, the experiments were repeated 10,000 times for statistical significance. The boards considered for the experiments are reported in Table 1, providing technical specifications.

Raspberry Pi 4 is about an order of magnitude smaller than the results obtained by the reference computer. Considering the MCU set, the best inference time is $41.16 \pm 2.9 \mu s$ reached by Portenta H7 (core M4). When analyzing the maximum times, to avoid high out-of-range values of the inference time, a washout operation was carried out to eliminate possible initial fluctuations in the execution of the first 100 tests. The same behavior was found for the GRU model (showed in Table 3). Clearly, the inference time increases as the hardware characteristics degrade, using the C code generated by NeuralCasting. The result obtained with the Raspberry Pi 4 is approximately an order of magnitude lower than the results from the reference computer. Among the MCU set, the best inference time is $211.60 \pm 3.86 \mu s$, achieved by the Portenta H7 (core M7). Furthermore, for all MCUs tested, it was not possible to implement the ONNX runtime version of the algorithms due to incompatibilities.

In the case of the x86_64 experiments, the standard deviation is relatively high compared to the mean due to the distribution of the measurements. In fact, out of 100,000 experiments, 99,989 measurements were in the order of 10^{-7} , 6 in the order of 10^{-6} , 4 in the order of 10^{-5} , and 1 in the order of 10^{-4} . Such sporadic outliers influence the standard

deviation considerably, inflating it by several orders of magnitude. A similar discussion concerns the measurements of the Raspberry Pi 4.

A comparison between NeuralCasting and TensorFlow Lite for GRU model deployment across various hardware platforms reveals notable differences in performance and stability. NeuralCasting consistently demonstrates lower and more stable inference latencies, with minimal variability across runs. In contrast, TensorFlow Lite (see Table 4), although achieving lower average inference times on some platforms (e.g., Portenta H7 and Arduino Nano 33 BLE), suffers from extremely high variability and significantly larger maximum latencies, often reaching up to $1000 \mu s$. This behavior indicates that while TensorFlow Lite may offer faster inference in optimal cases, it lacks the predictability required for real-time applications, where consistent timing is critical. On microcontrollers like the M5StickC Plus and Arduino Nano 33 BLE, NeuralCasting proves more reliable with stable performance, whereas TensorFlow Lite exhibits erratic latencies that could compromise system responsiveness. Overall, if deterministic behavior and low maximum latency are essential, NeuralCasting emerges as the preferred solution, whereas TensorFlow Lite may be suitable

for scenarios where occasional latency spikes are acceptable in exchange for faster average execution.

Furthermore, when comparing NeuralCasting and TensorFlow Lite, it is evident that TensorFlow Lite generally requires higher amounts of both RAM and Flash memory across all platforms for both MLP and GRU models (see Table 5). Specifically, TensorFlow Lite consistently consumes more Flash, reaching up to 0.58 MB on ESP32-PICO-D4 for both models, while NeuralCasting maintains significantly lower Flash usage, with peaks of only 0.27 MB for the GRU model on the same platform. Similarly, TensorFlow Lite's RAM usage remains slightly higher than NeuralCasting, with values such as 0.16 MB versus 0.15 MB on STM32H747XI-M7 for GRU, and 0.13 MB versus 0.12 MB on nRF52840 for MLP. These differences highlight NeuralCasting's advantage in minimizing memory footprint, making it a more efficient option for deeply embedded systems where limited memory resources are a critical constraint.

In Figure 5, a bar plot shows the result previously discussed, in this way was possible to highlight the inference time distribution among the selected devices.

This approach leverages a custom TinyML compiler to enable energy-efficient AI models on low-power edge devices, ensuring real-time detection of microplastics with minimal latency crucial for timely environmental monitoring.

V. CONCLUSION

In this study, we addressed the critical issue of microplastic detection in fluids, showing how technology can deal with the attempt to solve a topic relevant to environmental conservation and public health, as suggested by the 17 Sustainable Development Goals adopted by UN members. We generated a synthetic dataset and evaluated two NN models (MLP and GRU) using a Neural Architecture Search strategy named Grid Search to identify the optimal configurations. Our proposed framework, NeuralCasting, compiles neural networks directly into native C code, eliminating the need for a runtime engine. While ONNX Runtime and other state-of-the-art frameworks offer advanced optimizations, NeuralCasting prioritizes efficiency for bare-metal edge devices. It improves latency, memory usage, energy efficiency, and portability across diverse hardware platforms, making it ideal for small models where extensive parallelization is unnecessary.

Future work should investigate tiling and vectorization techniques optimized for diverse hardware architectures to further improve performance. Additionally, extending the proposed approach to more complex architectures will require exploring its scalability through the integration of more efficient NAS strategies—such as multi-objective or surrogate-based methods—that can effectively navigate larger search spaces while maintaining manageable computational overhead. Recent advancements in Edge AI have introduced emerging techniques such as attention-based [37] mechanisms and transformer models adapted for TinyML [38], which offer promising avenues for further

improving microplastic detection on resource-constrained devices. These directions not only align with the growing demand for sustainable environmental monitoring but also provide a foundation for deploying sophisticated AI models directly on edge devices. Efficient deployment on edge devices enables real-time monitoring and rapid response to microplastic contamination, which is critical for timely environmental protection efforts. Extending compatibility and evaluating performance on emerging platforms such as RISC-V and next-generation microcontrollers equipped with AI acceleration will further strengthen this capability. These platforms offer promising features, including energy-efficient parallelism and dedicated machine learning instruction sets, which can significantly enhance the overall performance and scalability of the proposed TinyML pipeline.

ACKNOWLEDGMENT

(Alessandro Cerioli and Lorenzo Petrosino contributed equally to this work.)

DATA AVAILABILITY

The code, data, and documentation relating to the experiments are available via the open source repository: <https://github.com/alecerio/microplastic>

REFERENCES

- [1] L. Lebreton, M. Egger, and B. Slat, "A global mass budget for positively buoyant macroplastic debris in the ocean," *Sci. Rep.*, vol. 9, no. 1, pp. 1–10, Sep. 2019.
- [2] J. N. Hahladakis, C. A. Velis, R. Weber, E. Iacovidou, and P. Purnell, "An overview of chemical additives present in plastics: Migration, release, fate and environmental impact during their use, disposal and recycling," *J. Hazardous Mater.*, vol. 344, pp. 179–199, Feb. 2018.
- [3] M. Cole, P. Lindeque, C. Halsband, and T. S. Galloway, "Microplastics as contaminants in the marine environment: A review," *Mar. Pollut. Bull.*, vol. 62, no. 12, pp. 2588–2597, Dec. 2011.
- [4] K. Duis and A. Coors, "Microplastics in the aquatic and terrestrial environment: Sources (with a specific focus on personal care products), fate and effects," *Environ. Sci. Eur.*, vol. 28, no. 1, pp. 1–25, Dec. 2016.
- [5] A. Ragusa, A. Svelato, C. Santacroce, P. Catalano, V. Notarstefano, O. Carnevali, F. Papa, M. C. A. Rongioletti, F. Baiocco, S. Draghi, E. D'Amore, D. Rinaldo, M. Matta, and E. Giorgini, "Plasticenta: First evidence of microplastics in human placenta," *Environ. Int.*, vol. 146, Jan. 2021, Art. no. 106274.
- [6] *The 17 Goals-Sustainable Development*, Sustainable Development Goals, USA, 2024. Accessed: Jul. 11, 2024. [Online]. Available: <https://sdgs.un.org/goals>
- [7] A. Baruah, A. Sharma, S. Sharma, and R. Nagraik, "An insight into different microplastic detection methods," *Int. J. Environ. Sci. Technol.*, vol. 19, no. 6, pp. 5721–5730, Jun. 2022.
- [8] M. Kedzierski, M. Falcou-Préfol, M. E. Kerros, M. Henry, M. L. Pedrotti, and S. Bruzard, "A machine learning algorithm for high throughput identification of FTIR spectra: Application on microplastics collected in the Mediterranean Sea," *Chemosphere*, vol. 234, pp. 242–251, Nov. 2019.
- [9] V. H. da Silva, F. Murphy, J. M. Amigo, C. Stedmon, and J. Strand, "Classification and quantification of microplastic (< 100 μm) using fpa-ir imaging system and machine learning," *Anal. Chem.*, vol. 92, no. 20, pp. 13724–13733, 2020.
- [10] X. Yan, Z. Cao, A. Murphy, and Y. Qiao, "An ensemble machine learning method for microplastics identification with FTIR spectrum," *J. Environ. Chem. Eng.*, vol. 10, no. 4, Aug. 2022, Art. no. 108130.
- [11] K. S. Awaisi, Q. Ye, and S. Sampalli, "A survey of industrial AIoT: Opportunities, challenges, and directions," *IEEE Access*, vol. 12, pp. 96946–96996, 2024.

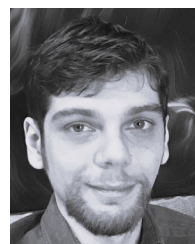
- [12] P. P. Ray, "A review on TinyML: State-of-the-art and prospects," *J. King Saud Univ.- Comput. Inf. Sci.*, vol. 34, no. 4, pp. 1595–1623, Apr. 2022.
- [13] Y. Abadade, A. Temouden, H. Bamoumen, N. Benamar, Y. Chtouki, and A. S. Hafid, "A comprehensive survey on TinyML," *IEEE Access*, vol. 11, pp. 96892–96922, 2023.
- [14] R. Immonen and T. Hämäläinen, "Tiny machine learning for resource-constrained microcontrollers," *J. Sensors*, vol. 2022, Nov. 2022, Art. no. 743702.
- [15] M. Alaa, A. A. Zaidan, B. B. Zaidan, M. Talal, and M. L. M. Kiah, "A review of smart home applications based on Internet of Things," *J. Netw. Comput. Appl.*, vol. 97, pp. 48–65, Nov. 2017.
- [16] V. Tsoukas, E. Boumpa, G. Giannakas, and A. Kakarountas, "A review of machine learning and TinyML in healthcare," in *Proc. 25th Pan-Hellenic Conf. Informat.*, Nov. 2021, pp. 69–73.
- [17] Y. Y. Siang, M. R. Ahamd, and M. S. Z. Abidin, "Anomaly detection based on tiny machine learning: A review," *Open Int. J. Informat.*, vol. 9, no. 2, pp. 67–78, 2021.
- [18] T. Flores, M. Silva, P. Andrade, J. Silva, I. Silva, E. Sisinni, P. Ferrari, and S. Rinaldi, "A TinyML soft-sensor for the Internet of Intelligent Vehicles," in *Proc. IEEE Int. Workshop Metrology Automot. (MetroAutomotive)*, Jul. 2022, pp. 18–23.
- [19] A. Gkogkidis, V. Tsoukas, S. Papafotikas, E. Boumpa, and A. Kakarountas, "A TinyML-based system for gas leakage detection," in *Proc. 11th Int. Conf. Modern Circuits Syst. Technol. (MOCAS)*, Jun. 2022, pp. 1–5.
- [20] H. Cheng, M. Zhang, and J. Q. Shi, "A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 10558–10578, Dec. 2024.
- [21] U. Kulkarni, A. S. Hosamani, A. S. Masur, S. Hegde, G. R. Vernekar, and K. Siri Chandana, "A survey on quantization methods for optimization of deep neural networks," in *Proc. Int. Conf. Autom., Comput. Renew. Syst. (ICACRS)*, Dec. 2022, pp. 827–834.
- [22] W. Niu, J. Guan, Y. Wang, G. Agrawal, and B. Ren, "DNNFusion: Accelerating deep neural networks execution with advanced operator fusion," in *Proc. 42nd ACM SIGPLAN Int. Conf. Program. Lang. Design Implement.*, Jun. 2021, pp. 883–898.
- [23] TensorFlow. *Tensorflow Lite Micro*. Accessed: Sep. 11, 2024. [Online]. Available: <https://github.com/tensorflow/tflite-micro>
- [24] *Tvm: An End to End Machine Learning Compiler Stack*, 2024, Apache Software Found., Forest Hill, MD, USA, 2024.
- [25] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proc. 13th USENIX Symp. Operating Syst. Design Implement. (OSDI)*, 2018, pp. 578–594.
- [26] Microsoft. (2023). *ONNX Runtime*. [Online]. Available: <https://github.com/microsoft/onnxruntime>
- [27] T. Thammasanya, S. Patiam, E. Rodcharoen, and P. Chotikarn, "A new approach to classifying polymer type of microplastics based on faster-RCNN-FPN and spectroscopic imagery under ultraviolet light," *Sci. Rep.*, vol. 14, no. 1, p. 3529, Feb. 2024.
- [28] O. Guselnikova, A. Trelin, Y. Kang, P. Postnikov, M. Kobashi, A. Suzuki, L. K. Shrestha, J. Henzie, and Y. Yamauchi, "Pretreatment-free SERS sensing of microplastics using a self-attention-based neural network on hierarchically porous Ag foams," *Nature Commun.*, vol. 15, no. 1, p. 4351, May 2024.
- [29] G. Lee and K. Jhang, "Neural network analysis for microplastic segmentation," *Sensors*, vol. 21, no. 21, p. 7030, Oct. 2021.
- [30] *Open Neural Network Exchange (ONNX)*. Accessed: Nov. 7, 2024. [Online]. Available: <https://github.com/onnx/onnx>
- [31] Alessandro Cerioli. (2024). *Neuralcasting: Front-end Compiler for Onnx*. [Online]. Available: <https://github.com/alecchio/NeuralCasting>
- [32] A. Cerioli, R. Miccini, C. Laroche, T. Piechowiak, L. Pezzarossa, J. Sparsø, and M. Schoeberl, "NeuralCasting: A front-end compilation infrastructure for neural networks," in *Proc. 11th Int. Conf. Internet Things, Syst., Manage. Secur. (IOTSMS)*, Sep. 2024, pp. 161–168.
- [33] R. Miccini, A. Cerioli, C. Laroche, T. Piechowiak, J. Sparsø, and L. Pezzarossa, "Towards a tailored mixed-precision sub-8-bit quantization scheme for gated recurrent units using genetic algorithms," 2024, *arXiv:2402.12263*.
- [34] P. Ren, Y. Xiao, X. Chang, P.-Y. Huang, Z. Li, X. Chen, and X. Wang, "A comprehensive survey of neural architecture search: Challenges and solutions," *ACM Comput. Surv.*, vol. 54, no. 4, pp. 1–34, May 2022.
- [35] D. Sasso, M. Merone, E. Nicolai, L. Vollero, and A. Sabatini, "A benchmarking on optofluidic microplastic pattern recognition: A systematic comparison between statistical detection models and ML-based algorithms," *IEEE Access*, vol. 12, pp. 20091–20102, 2024.
- [36] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," 2014, *arXiv:1406.1078*.
- [37] P. Wiese, G. Islamoglu, M. Scherer, L. Macan, V. J. B. Jung, A. Burrello, F. Conti, and L. Benini, "Toward attention-based TinyML: A heterogeneous accelerated architecture and automated deployment flow," *Proc. IEEE Design Test*, early access, Jan. 8, 2025, doi: [10.1109/MDAT.2025.3527371](https://doi.org/10.1109/MDAT.2025.3527371).
- [38] T. Suwannaphong, F. Jovan, I. Craddock, and R. McConville, "Optimising TinyML with quantization and distillation of transformer and mamba models for indoor localisation on edge devices," *Sci. Rep.*, vol. 15, no. 1, p. 10081, Mar. 2025.



ALESSANDRO CERIOLI received the master's degree in sound and music computing from Aalborg University, in 2021, specializing in real-time sound processing. He is currently pursuing the Industrial Ph.D. degree with the Technical University of Denmark (DTU) and Jabra, a renowned Danish company specializing in advanced audio and communication products. Successively, he was with GN Hearing, a leading company in the field of medical devices. His Ph.D. is funded by the European research program Convolv as part of the Horizon project. His research interests include neural network compression using TinyML techniques and compilers for neural networks for efficient deployment of AI applications on edge devices.



LORENZO PETROSINO received the master's degree in biomedical engineering from Università Campus Bio-Medico di Roma, in 2020, where he is currently pursuing the Ph.D. degree in science and engineering for humans and the environment. His current research interests include machine and deep learning, edge computing, DLT and its interaction with the IoT, and also decision support system for diabetes.



DANIELE SASSO received the master's degree in cognitive neuroscience from Sapienza University of Rome, in 2020. He is currently pursuing the Ph.D. degree in AI with the Research Unit of Computer Systems and Bioinformatics, Department of Engineering, Università Campus Bio-Medico di Roma. His current research interests include machine learning, deep learning, reinforcement learning, and AI applications to biomedical and agronomical fields.



CLÉMENT LAROCHE received the Ph.D. degree from Télécom ParisTech, in 2017. He is currently a Research Scientist in machine learning and audio processing with GN Audio. His research interests include deep learning for speech enhancement, neural network compression, and edge AI for audio applications.



TOBIAS PIECHOWIAK (Member, IEEE) received the Bachelor of Science (B.Sc.) degree in mathematics from the University of Heidelberg, in 2002, the Master of Science (M.Sc.) degree in electrical engineering from the Technical University of Denmark, in 2004, and the Doctor of Philosophy (Ph.D.) degree in electrical engineering, specializing in digital signal processing. In 2010, he joined GN Store Nord A/S as a Research Scientist. His primary research and development interests include algorithms for hearing aids, with expertise in models of the auditory system, compression, noise reduction, fitting algorithms, and medical statistics. Recently, he transitioned to Jabra to apply AI methods in noise suppression, echo suppression (Speak2), speech enhancement, chipsets for edge computing, and the management of external projects.



LUCA PEZZAROSSA (Member, IEEE) received the Ph.D. degree from the Technical University of Denmark, in 2018. He is currently an Associate Professor of computer engineering with the Technical University of Denmark. His research interests include reconfigurable systems, real-time cyber-physical/embedded systems, and embedded AI for audio applications.



MARIO MERONE (Member, IEEE) received the Ph.D. degree in bioengineering and bioscience, in 2017. He is currently an Assistant Professor with Università Campus Bio-Medico di Roma. He has extensive experience in the field of artificial intelligence (machine learning and deep learning for time-series analysis). His research has led to 40 publications, all focused on AI for healthcare. He is the Scientific Leader of several projects involving the application of artificial intelligence in the medical field. He is a founding partner of an innovative start-up called BPCOMEDIA S. R. L., focused on the technology transfer of research results. He is a member of the IEEE Computer Society, the IEEE Computational Life Sciences, and the IEEE Sensors Council.



LUCA VOLLERO (Member, IEEE) is currently an Associate Professor of computer science with Università Campus Bio-Medico di Roma. His research activities fall in the areas of signal processing and digital imaging, AI, distributed systems, and embedded systems optimization. He is a member of the IEEE Computer Society, the IEEE Communication Society, ACM, and SIAM. He is the Co-Founder of the innovative startup Heremos, operating in the field of remote monitoring of frail persons and patients.



ANNA SABATINI (Member, IEEE) received the master's degree in biomedical engineering and the Ph.D. degree in science and engineering for humans and the environment from Università Campus Bio-Medico di Roma, in 2017 and 2021, respectively. She is currently a Researcher in computer systems and bioinformatics.

...